UNIVERSIDAD PERUANA CAYETANO HEREDIA

PUCP

INGENIERÍA BIOMÉDICA

CICLO: 2026 - II

ENTREGABLE N°10

CASO CLÍNICO 3

Traumatismo craneoencefálico

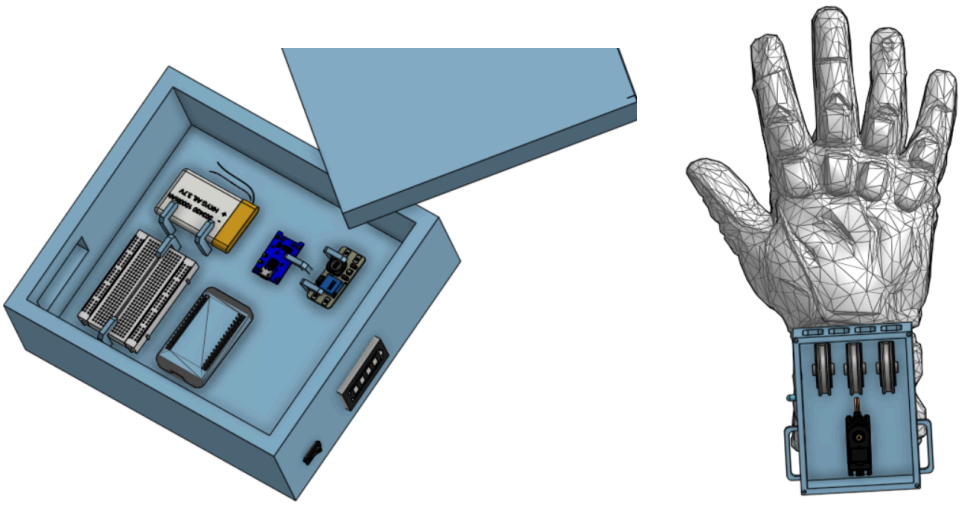
Neyra Quezada, Gabriela Estefanía
Lazaro Abanto, Diego David
Muñoz Huatuco, Kiara Pierina
Martínez Aquino, Valeria Jimena
Ramos Guerra, Alejandro Josue
Hoyos Puente, Abner Mihails

Curso: Fundamentos de Biodiseño

Verificación preliminar del diseño

A. Mecánica y fabricación

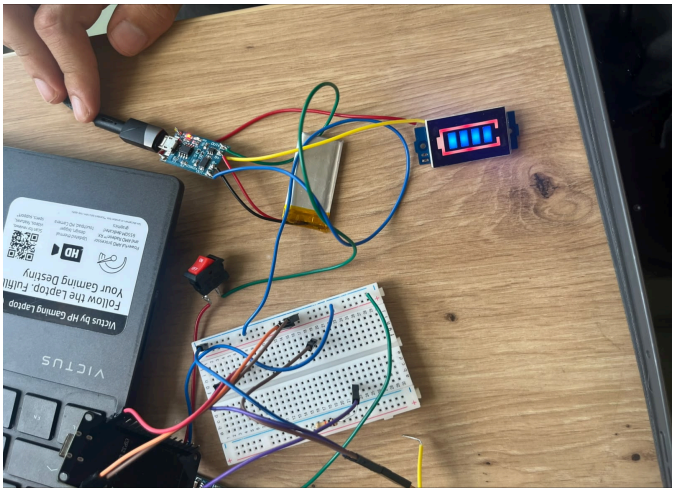
Se realizó la verificación dimensional y funcional de los componentes fabricados frente a la geometría original del modelo CAD. Las pruebas comprobaron el ajuste adecuado entre las interfaces, el correcto acoplamiento de las piezas impresas y la fluidez del movimiento en las articulaciones móviles, asegurando la ausencia de atascamientos o holguras excesivas durante el ensamble.



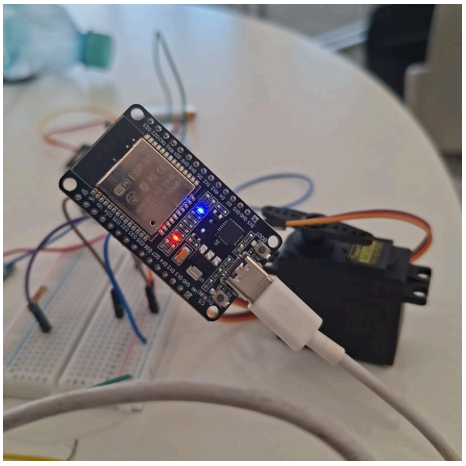
B. Electrónica y sensores

Para el apartado de Electrónica y sensores, se validó la continuidad del cableado, la estabilidad de las líneas de alimentación y la respuesta individual de cada componente sensorial y actuador. Las pruebas operativas confirmaron la correcta lectura de las variables físicas, la integridad de las señales analógicas y digitales enviadas al microcontrolador y la activación precisa de los elementos de salida bajo cargas de trabajo controladas.

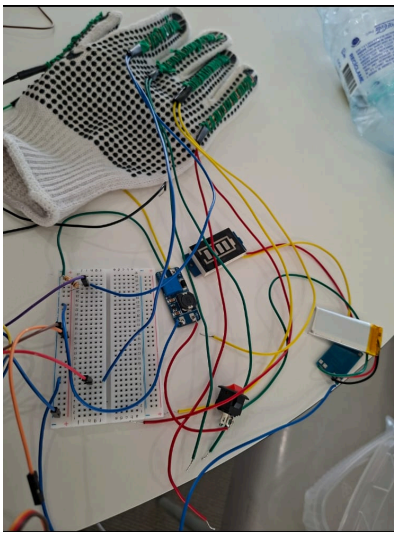
Conexiones e indicador de batería funcionando correctamente:

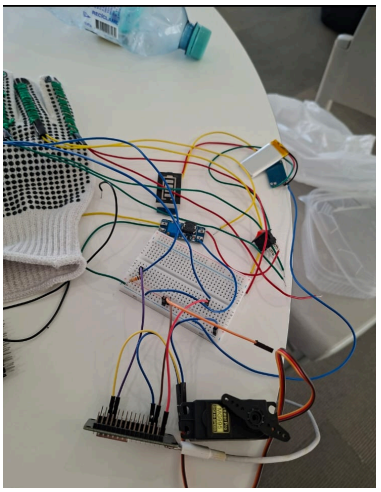


Funcionamiento del ESP32:

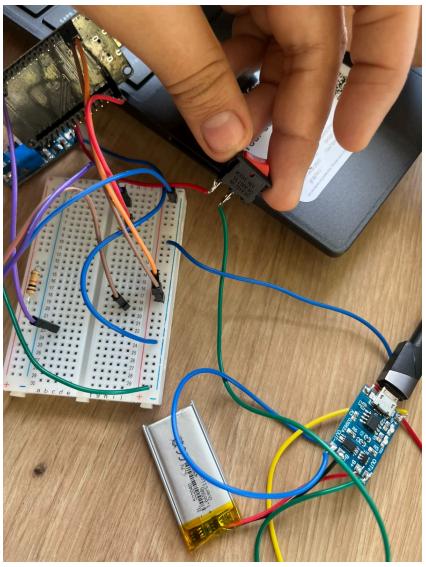


Guante soldado con sensores flex:

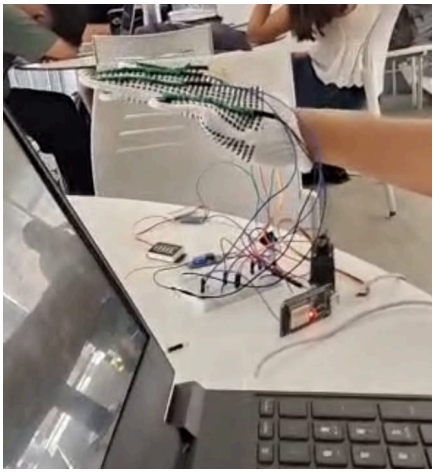




Conexiones Rocker Switch:



Prueba del Dispositivo:



Codigo funcionando con Flex:

```
ctura:400 baseline:430 caida:30 angulo:28
ctura:399 baseline:430 caida:31 angulo:25
ctura:398 baseline:430 caida:32 angulo:22
ctura:398 baseline:430 caida:32 angulo:19
ctura:399 baseline:430 caida:31 angulo:16
ctura:399 baseline:430 caida:31 angulo:13
ctura:399 baseline:430 caida:31 angulo:10
ctura:398 baseline:430 caida:32 angulo:7
```

Tabla de Conexiones

N.°	Origen	Destino
1. Etapa de alimentación		
1	Batería LiPo 3.7 V (+)	B+ del módulo TP4056
2	Batería LiPo 3.7 V (–)	B– del módulo TP4056
3	OUT+ del TP4056	Terminal del interruptor (rocker switch)
4	Terminal del interruptor	VIN+ del MT3608
5	OUT– del TP4056	Riel GND
6	VIN– del MT3608	Riel GND
7	VOUT– del MT3608	Riel GND
8	VOUT+ del MT3608 (ajustado a 5.0 V)	Riel +5 V
2. Microcontrolador ESP32		
1	VIN	Riel +5 V
2	GND	Riel GND
3	3V3	Terminal 1 del sensor flex
4	D34 (ADC1, entrada analógica)	Nodo del divisor de tensión
5	D13 (salida PWM)	Cable de señal del servomotor
3. Sensor flex y divisor de tensión		
1	Sensor flex, terminal 1	Pin 3V3 del ESP32
2	Sensor flex, terminal 2	Nodo del divisor (común con D34 y la resistencia)
3	Resistencia de 10 kΩ, pata 1	Nodo del divisor
4	Resistencia de 10 kΩ, pata 2	Riel GND
	$V_{D34} = 3.3\ V \cdot 10\ k\Omega / (R_{flex} + 10\ k\Omega)$. Al flexionar el sensor aumenta R_{flex}	
4. Servomotor MG995		
1	Rojo (+)	Riel +5 V
2	Marrón o negro (GND)	Riel GND
3	Naranja o amarillo (señal)	Pin D13 del ESP32

C. Programación y control

Se comprobó la ejecución del código en relación con la arquitectura lógica prevista en el diagrama de flujo. Se verificó el tiempo de respuesta del algoritmo ante los eventos de entrada, el filtrado de datos en tiempo real y la transición adecuada entre los distintos estados del sistema, garantizando un control coordinado y estable de los actuadores.

Explicación del funcionamiento del código del guante asistivo

El código implementado en el ESP32 permite controlar la extensión de los dedos mediante un sensor flex y un servomotor MG995. Su funcionamiento se divide en seis módulos:

a. Configuración de parámetros

Se establecen los pines de conexión del sensor flex (GPIO 34) y del servomotor (GPIO 13). Asimismo, se definen los ángulos de reposo (0°) y tracción (90°), además de los umbrales de activación y desactivación del sistema.

- PIN_FLEX = 34: entrada analógica (ADC1) donde se lee el divisor del flex.
- PIN_SERVO = 13: salida PWM hacia el MG995.
- Servo: ANGULO_REPOSO = 0° (hilo flojo) y ANGULO_X = 90° (tracción).
- Sensor: 16 muestras por promedio, UMBRAL_ON = 80 y UMBRAL_OFF = 40.
- Todo en constantes al inicio para ajustar sin tocar la lógica.

```
const int PIN_FLEX  = 34;
const int PIN_SERVO = 13;

const int ANGULO_REPOSO = 0;
const int ANGULO_X      = 90;
const unsigned long
  INTERVALO_PASO_MS = 25;

const int MUESTRAS   = 16;
const int UMBRAL_ON  = 80;
const int UMBRAL_OFF = 40;
```

b. Lectura del sensor flex

Se implementa la función leerFlex(), que realiza 16 lecturas analógicas del sensor y calcula su promedio. Esto permite reducir las variaciones ocasionadas por el ruido en las mediciones.

- leerFlex() devuelve el promedio de 16 lecturas del ADC.
- Promediar reduce el ruido, que en el ESP32 es alto.
- El ADC trabaja a 12 bits (0 a 4095) con atenuación de 11 dB (hasta ~3.3 V).
- Divisor: 3V3, flex, D34 y 10 kΩ a GND. Al doblar el flex, el voltaje baja.

```
int leerFlex() {
  long suma = 0;
  for (int i = 0; i < MUESTRAS; i++) {
    suma += analogRead(PIN_FLEX);
    delayMicroseconds(200);
  }
  return (int)(suma / MUESTRAS);
}
```

c. Inicialización del sistema

Al iniciar, el sistema espera 3 segundos para permitir que el usuario se coloque el guante. Posteriormente, registra durante 2 segundos una medición de referencia (baseline) con la mano relajada. Finalmente, activa el servomotor en su posición inicial.

- Espera de 3 s para ponerse el guante (LED parpadeo lento).
- Durante 2 s toma la referencia (baseline) con la mano relajada (LED rápido).
- Recién después activa el servo, para no sumar su pico de corriente al arranque del ESP32.
- El servo parte en ANGULO_REPOSO.

```
parpadear(3000, 1000);   // espera

// referencia, mano relajada
while (millis() - inicio < 2000) {
  suma += leerFlex(); n++;
}
baseline = suma / n;

// recién ahora el servo
servo.attach(PIN_SERVO, 500, 2400);
servo.write(ANGULO_REPOSO);
```

d. Detección de movimiento mediante histéresis

El programa compara la lectura actual del sensor con el valor de referencia. Cuando la diferencia alcanza el umbral de activación (80), se inicia la tracción. Cuando desciende hasta el umbral de desactivación (40), se libera. Este mecanismo evita movimientos involuntarios del servomotor debido a fluctuaciones en las lecturas.

- caida = baseline – lectura: cuánto se flexionó el dedo respecto al reposo.
- Si caida ≥ UMBRAL_ON (80), se activa la tracción.
- Si caida ≤ UMBRAL_OFF (40), se suelta el hilo.
- Entre 40 y 80 mantiene el estado: evita que el servo vibre por ruido.

```
int lectura = leerFlex();
int caida = baseline - lectura;

if (!traccionActiva && caida >= UMBRAL_ON) {
  traccionActiva = true;
  anguloObjetivo = ANGULO_X;
} else if (traccionActiva &&
           caida <= UMBRAL_OFF) {
  traccionActiva = false;
  anguloObjetivo = ANGULO_REPOSO;
}
```

e. Control del movimiento del servomotor

El servomotor realiza un desplazamiento progresivo de 1° cada 25 ms hasta alcanzar el ángulo objetivo. De esta manera, se evita un movimiento brusco durante la tracción o liberación del mecanismo.

- El servo no salta: avanza 1° cada 25 ms hacia el ángulo objetivo.
- De 0° a 90° tarda unos 2.3 s, lo que protege al paciente y a la fuente.
- El LED integrado queda encendido fijo mientras hay tracción.
- Se ajusta con PASO_GRADOS e INTERVALO_PASO_MS.

```
digitalWrite(PIN_LED,
  traccionActiva ? HIGH : LOW);

if (millis() - ultimoPaso >= 25) {
  ultimoPaso = millis();
  if (anguloActual < anguloObjetiv
    anguloActual = min(
      anguloActual + PASO_GRADOS,
      anguloObjetivo);
    servo.write(anguloActual);
  }
}
```

f. Motorización y ajustes

Finalmente, mediante el Monitor Serie a 115200 baudios, se visualizan los valores del sensor, la referencia, la diferencia calculada y el ángulo del servomotor. Estos parámetros permiten realizar ajustes en la sensibilidad y velocidad de respuesta del sistema.

- Con USB, el Monitor Serie (115200) muestra lectura, baseline, caída y ángulo.
- UMBRAL_ON: más bajo = más sensible (no menos de ~30, por el ruido).
- ANGULO_X: sale de la geometría del hilo (hilo a jalar ÷ radio del brazo del servo).
- INTERVALO_PASO_MS: velocidad del movimiento.

```
lectura:429 baseline:430 caida:1  angulo:0
lectura:398 baseline:430 caida:32 angulo:7

// Ajustes principales
UMBRAL_ON  = 80
UMBRAL_OFF = 40
ANGULO_X   = 90
INTERVALO_PASO_MS = 25
```